

OWASP Juice Shop Security Assessment

Project:	DevSecOps Pipeline & Baseline Security Review	Prepared For:	Engineering Manager
Target System:	OWASP Juice Shop Application	Prepared By:	Sandeep Sain

EXECUTIVE SUMMARY

The current OWASP Juice Shop application is built with intentional vulnerabilities and operates without automated security checks during its build and deployment pipeline. Pushing code without continuous security scanning exposes production environments to secret leaks, vulnerable third-party packages, code injection vulnerabilities, and root-level container compromises. Addressing these issues early in the delivery lifecycle is critical to preventing data breaches, service disruption, and financial or compliance fallout.

TOP 5 SECURITY RISKS IDENTIFIED

RISK DESCRIPTION	SEVERITY	RISK JUSTIFICATION
Vulnerable Third-Party Dependencies	CRITICAL	Known vulnerabilities in open-source dependencies allow attackers to breach the system without exploiting application-specific code.
Hardcoded Secrets & Credentials	CRITICAL	Committed API tokens, private keys, or passwords provide unauthorized parties immediate access to databases and cloud resources.
Application Code Vulnerabilities	HIGH	Insecure code patterns enable injection attacks, authentication bypasses, and unauthorized exposure of application data.
Insecure Container Configuration	HIGH	Running containers as the root user or using unpatched base images drastically expands the blast radius if a container is compromised.
Missing CI/CD Automated Security Gates	MEDIUM	Builds fail to block severe security findings, allowing insecure code to flow seamlessly into active environments.

CRITICAL & HIGH RISK ATTACK SCENARIOS

1. Vulnerable Third-Party Dependencies (Critical)

An attacker identifies public CVEs linked to outdated node packages in the application build. By crafting an exploit against a known dependency flaw, the attacker executes arbitrary commands or extracts application memory remotely.

Business Impact: Customer data exposure, application downtime, emergency response costs, and brand degradation.

2. Hardcoded Secrets or Credentials (Critical)

A private key, API token, or database credential is accidentally committed to source control. An attacker accessing the repository extracts these keys and gains direct administrative control over internal services or cloud storage.

Business Impact: Complete cloud account compromise, massive data theft, and regulatory non-compliance penalties.

3. Application Code Vulnerabilities (High)

An attacker audits the web application and discovers unvalidated inputs or flawed authentication logic. Exploiting these flaws allows them to bypass login controls, manipulate SQL queries, or extract other users' records.

Business Impact: Mass account takeover, fraudulent financial transactions, and loss of customer trust.

4. Insecure Container Configuration (High)

An attacker compromises the web application process running inside a container. Because the container process runs with root privileges and contains system tools, the attacker easily pivots into the underlying host network.

Business Impact: Lateral network movement, host system takeover, and extensive incident recovery downtime.

Recommended Implementation Priority

Our primary objective is to embed automated security controls directly into the CI/CD pipeline so that security defects are caught before deployment:

- **Static Application Security Testing (SAST):** Scan application code on every pull request to catch logic and injection flaws early.
- **Software Composition Analysis (SCA):** Automatically scan open-source libraries for known CVEs and fail builds with Critical/High findings.
- **Secrets Detection:** Enforce git pre-commit/pipeline secret scanning to prevent credentials from entering the codebase.
- **Hardened Container Dockerfiles:** Build application images on minimal non-root base images (e.g., `node:alpine`) running as `USER node`.
- **Strict Pipeline Gating:** Automatically fail builds and block merges when High or Critical vulnerabilities are flagged.